Homework 10 Solutions

Prerith Arunkumar (prerith2)

A bias-variance simulation for random-forest node size and `mtry`, OOB tuning of a classification forest on PCA-reduced MNIST digits, and a plan for a random-forest analysis skill.

Use of AI tools. I used Claude (Claude Code) to draft the `R` code and exposition for Questions 1 and 2 and to run it. Every number below comes from the code output shown in this report, and I checked the numbers quoted in the text against it. Question 3 is my own writing; I asked Claude to explain what the question asks and how an answer could be structured, but the paragraph and the ten instructions are mine.

All forests use the `RLT` package (version 6.1.1) with ordinary random forests (`reinforcement = FALSE`, the default), bootstrap sampling with replacement, and `nmin` as the node-size setting.

```
suppressPackageStartupMessages(library(RLT))
library(MASS)
ncores <- 1   # this RLT build has no OpenMP, so fits run single-threaded
```

Question 1: Bias-variance trade-off simulation

(a) Simulation

The test points are drawn once. In each of 100 repetitions a new training set of 200 observations is generated, and the same training set is used for all nine distinct settings (`mtry = 3`, node size 10 appears in both comparisons and is fitted once). Each forest has 500 trees, and `resample.prob = 1` with replacement draws 200 observations per tree. For setting s and test point $\mathbf{x}_j$, with predictions $\hat{f}_r(\mathbf{x}_j)$ over repetitions r, squared bias is $\left(\bar{\hat{f}}(\mathbf{x}_j) - f(\mathbf{x}_j)\right)^2$ and variance is $\frac{1}{100}\sum_r \left(\hat{f}_r(\mathbf{x}_j) - \bar{\hat{f}}(\mathbf{x}_j)\right)^2$; both are then averaged over the 200 test points.

```
seed <- 432
set.seed(seed)
p <- 5
Sigma <- matrix(0.3, p, p); diag(Sigma) <- 1
f_true <- function(x) x[, 1] + x[, 3] + x[, 5]

n_test <- 200; n_train <- 200; reps <- 100
x_test <- mvrnorm(n_test, rep(0, p), Sigma)
f_test <- f_true(x_test)

settings <- unique(rbind(
  data.frame(mtry = 3, nmin = c(1, 5, 10, 20, 30)),
  data.frame(mtry = 1:5, nmin = 10)
))
settings$key <- paste0("m", settings$mtry, "_n", settings$nmin)

fit_predict <- function(x, y, xnew, mtry, nmin, rprob, s) {
  fit <- RLT(x, y, model = "regression", ntrees = 500, mtry = mtry, nmin = nmin,
             resample.prob = rprob, resample.replace = TRUE,
             ncores = ncores, seed = s, verbose = FALSE)
```

```
  as.vector(predict(fit, xnew)$Prediction)
}

run_sim <- function(settings, rprob) {
  pred <- array(NA, c(reps, n_test, nrow(settings)), dimnames = list(NULL, NULL,
settings$key))
  for (r in 1:reps) {
    x <- mvrnorm(n_train, rep(0, p), Sigma)
    y <- f_true(x) + rnorm(n_train)
    for (k in seq_len(nrow(settings)))
      pred[r, , k] <- fit_predict(x, y, x_test, settings$mtry[k], settings$nmin[k], rprob,
seed + r)
  }
  pred
}

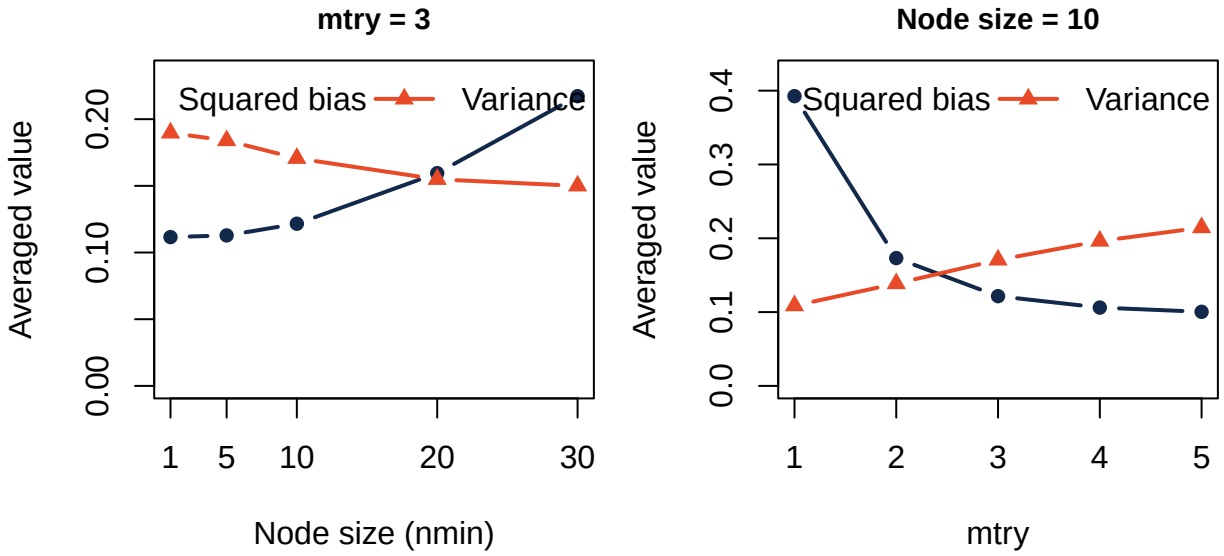
summarize_bv <- function(pred, settings) {
  m <- apply(pred, c(2, 3), mean)
  v <- apply(pred, c(2, 3), function(z) mean((z - mean(z))^2))
  cbind(settings[, c("mtry", "nmin")],
        bias2 = colMeans((m - f_test)^2), variance = colMeans(v))
}

pred200 <- run_sim(settings, rprob = 1)
bv <- summarize_bv(pred200, settings)
bv$total <- bv$bias2 + bv$variance
knitr::kable(bv, digits = 4, row.names = FALSE,
             col.names = c("mtry", "node size", "sq. bias", "variance", "bias^2 + var"))
```

mtry	node size	sq. bias	variance	bias^2 + var
3	1	0.1116	0.1897	0.3013
3	5	0.1128	0.1840	0.2968
3	10	0.1216	0.1707	0.2923
3	20	0.1595	0.1549	0.3144
3	30	0.2172	0.1500	0.3672
1	10	0.3925	0.1087	0.5012
2	10	0.1732	0.1387	0.3118
4	10	0.1062	0.1962	0.3023
5	10	0.1004	0.2145	0.3148

The random seed is 432 (set once before the test points are drawn; repetition r also passes `seed + r` to `RLT`).

```
plot_bv <- function(d, xvar, xlab, main) {
  d <- d[order(d[[xvar]]), ]
  yl <- c(0, max(d$bias2, d$variance) * 1.08)
  plot(d[[xvar]], d$bias2, type = "b", pch = 16, col = "#13294B", lwd = 2, ylim = yl,
       xlab = xlab, ylab = "Averaged value", main = main, xaxt = "n")
  axis(1, at = d[[xvar]])
  lines(d[[xvar]], d$variance, type = "b", pch = 17, col = "#E84A27", lwd = 2)
  legend("top", c("Squared bias", "Variance"), col = c("#13294B", "#E84A27"),
         pch = c(16, 17), lwd = 2, bty = "n", horiz = TRUE)
}
par(mfrow = c(1, 2), mar = c(4, 4, 2.2, 0.8), cex.main = 0.95)
plot_bv(subset(bv, mtry == 3), "nmin", "Node size (nmin)", "mtry = 3")
plot_bv(subset(bv, nmin == 10), "mtry", "mtry", "Node size = 10")
```



(b) Discussion

Large node size. With `mtry = 3`, squared bias rises from 0.112 at node size 1 to 0.217 at node size 30, while variance falls from 0.190 to 0.150. A node of up to 30 observations averages responses over a large region of $\mathbb{R}^5$, so each tree, and therefore the forest, is a coarse, flattened approximation of the linear function $x_1 + x_3 + x_5$: it shrinks predictions toward the mean at the edges of the predictor space. The estimate is very stable across training sets but systematically biased. Node size 10 gives the smallest squared bias plus variance in this comparison.

Large `mtry`. With node size 10, squared bias drops from 0.393 at `mtry = 1` to 0.100 at `mtry = 5`, and variance goes from 0.109 to 0.214. The benefit of a large `mtry` is that a split almost always has a signal predictor (X_1, X_3, or X_5) available, so splits are spent on directions that reduce error, lowering bias. With `mtry = 1`, about 40% of splits are forced onto X_2 or X_4; these are correlated with the signal, so they are not useless, but they are weaker. The drawback is that a large `mtry` makes trees more alike (`mtry = 5` is bagging), so averaging cancels less of their errors and the forest's variance tends to rise; with more noise predictors it would also make the trees more greedy and less diverse. In this example, with three of five predictors relevant, the bias reduction dominates, and `mtry = 3` has the smallest total.

Sampling 100 observations per tree. I expect the preferred node size to shift **smaller**. Each tree then sees half as many observations, so a given node size corresponds to a cell covering roughly twice as much of the predictor space, which increases bias. At the same time, the subsamples overlap less across trees, which decorrelates the trees and lowers the forest's variance. Both changes favour spending less smoothing on node size: the subsampling already supplies variance reduction, so bias becomes a larger share of the error, and that is reduced by smaller terminal nodes. A supplementary simulation with the same seed, test points, and `mtry = 3`, but `resample.prob = 0.5`, is below.

```
set.seed(seed + 1000)
settings_n <- subset(settings, mtry == 3)
pred100 <- run_sim(settings_n, rprob = 0.5)
bv100 <- summarize_bv(pred100, settings_n)
bv100$total <- bv100$bias2 + bv100$variance
cmp <- merge(bv[bv$mtry == 3, c("nmin", "total")], bv100[, c("nmin", "bias2", "variance",
"total")],
             by = "nmin", suffixes = c("_200", "_100"))
knitr::kable(cmp, digits = 4, col.names = c("node size", "total (200/tree)",
             "sq. bias (100/tree)", "variance (100/tree)", "total (100/tree)"))
```

node size	total (200/tree)	sq. bias (100/tree)	variance (100/tree)	total (100/tree)
1	0.3013	0.1578	0.1355	0.2933
5	0.2968	0.1617	0.1308	0.2924
10	0.2923	0.1861	0.1265	0.3126
20	0.3144	0.2856	0.1229	0.4084
30	0.3672	0.4367	0.1266	0.5632

With 100 observations per tree, the node size with the smallest total is 5, compared with 10 for 200 per tree. This is one simulation, and the difference between neighbouring node sizes is small relative to the totals, so the shift is a tendency rather than a guaranteed pattern.

Question 2: Tuning random forests for MNIST

```
mnist <- read.csv(gzfile("data/mnist-first2000.csv.gz"))
digits <- c(1, 6, 7)
train <- mnist[1:1000, ];  train <- train[train$Digit %in% digits, ]
test  <- mnist[1001:2000, ]; test <- test[test$Digit %in% digits, ]
pix <- setdiff(names(mnist), "Digit")

pca <- prcomp(train[, pix], center = TRUE, scale. = FALSE)
x_train <- pca$x[, 1:20]
x_test  <- predict(pca, test[, pix])[, 1:20]   # training means and loadings
y_train <- factor(train$Digit); y_test <- factor(test$Digit, levels = levels(y_train))
rbind(train = table(y_train), test = table(y_test))
```

```
        1   6   7
train 116  94 117
test  104 106 107
```

The first 20 components explain 77.1% of the training pixel variance.

(a) Default settings

```
rf_fit <- function(mtry = NULL, nmin = NULL, importance = FALSE) {
  args <- list(x = x_train, y = y_train, model = "classification", ntrees = 1000,
               resample.prob = 1, resample.replace = TRUE, importance = importance,
               ncores = ncores, seed = 2026, verbose = FALSE)
  if (!is.null(mtry)) args$mtry <- mtry
  if (!is.null(nmin)) args$nmin <- nmin
  do.call(RLT, args)
}
oob_err <- function(fit) mean(fit$Prediction != y_train)   # OOB predicted class vs. label

fit0 <- rf_fit()
c(mtry = fit0$parameters$mtry, nmin = fit0$parameters$nmin, OOB_error = oob_err(fit0))
```

```
       mtry        nmin   OOB_error
10.00000000  5.00000000  0.03975535
```

With RLT's defaults (`mtry` = 10, `nmin` = 5), the OOB misclassification rate is 0.0398 on 327 training digits.

(b) Tuning by OOB error

```
grid <- expand.grid(mtry = c(1, 2, 3, 4, 6, 8, 10, 15, 20), nmin = c(1, 2, 5, 10, 20))
grid$OOB <- sapply(seq_len(nrow(grid)), function(k) oob_err(rf_fit(grid$mtry[k],
grid$nmin[k])))
tab <- xtabs(OOB ~ nmin + mtry, grid)
knitr::kable(as.data.frame.matrix(tab), digits = 4,
             caption = "OOB misclassification rate (rows: node size nmin; columns: mtry).")
```

Table 1: OOB misclassification rate (rows: node size nmin; columns: mtry).

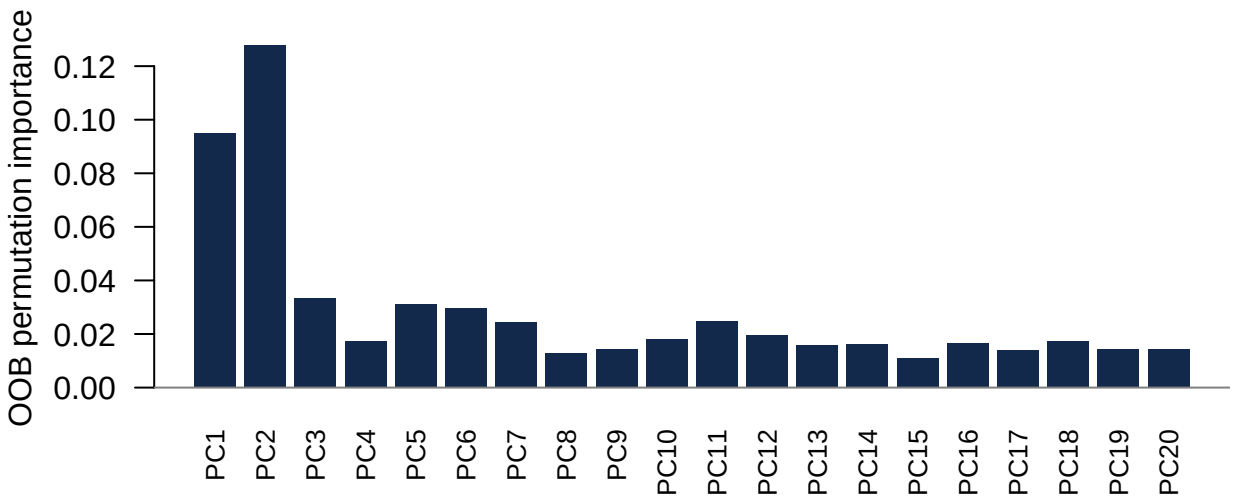
	1	2	3	4	6	8	10	15	20
1	0.0214	0.0336	0.0367	0.0398	0.0367	0.0398	0.0428	0.0428	0.0489
2	0.0214	0.0306	0.0367	0.0398	0.0398	0.0398	0.0428	0.0428	0.0489
5	0.0245	0.0306	0.0398	0.0398	0.0367	0.0398	0.0398	0.0428	0.0520
10	0.0275	0.0367	0.0367	0.0367	0.0398	0.0428	0.0428	0.0459	0.0520
20	0.0245	0.0398	0.0398	0.0367	0.0398	0.0428	0.0428	0.0428	0.0550

```
# Ties are broken toward the smaller mtry and then the larger node size (the simpler, less
greedy forest).
grid_o <- grid[order(grid$OOB, grid$mtry, -grid$nmin), ]
best <- grid_o[1, ]
best
```

```
   mtry nmin        OOB
10    1    2 0.02140673
```

The selected setting is `mtry` = 1 and node size 2, with OOB error 0.0214. The grid ranges from 0.0214 to 0.0550, i.e., 7 to 18 misclassified training digits, so many settings differ by only one or two observations and the choice among the top few is not sharply determined.

```
fit_best <- rf_fit(best$mtry, best$nmin, importance = TRUE)
vi <- setNames(as.vector(fit_best$VarImp), colnames(x_train))
par(mar = c(4, 4, 1, 0.5))
barplot(vi, las = 2, col = "#13294B", border = NA, ylab = "OOB permutation importance",
cex.names = 0.8)
abline(h = 0, col = "grey50")
```




```
round(sort(vi, decreasing = TRUE), 4)
```

```
   PC2    PC1    PC3    PC5    PC6   PC11    PC7   PC12   PC10   PC18    PC4
0.1277 0.0952 0.0332 0.0312 0.0296 0.0247 0.0244 0.0195 0.0182 0.0175 0.0174
  PC16   PC14   PC13    PC9   PC20   PC19   PC17    PC8   PC15
0.0168 0.0163 0.0160 0.0144 0.0143 0.0143 0.0140 0.0130 0.0111
```

Comments. The OOB errors are all small (the three digits are easy to separate in PC space), but there is a clear pattern in `mtry`: error increases almost steadily from `mtry = 1` (0.0214 to 0.0275 across node sizes) to `mtry = 20` (0.0489 to 0.0550), while node size changes the error by only a few observations within a column. With 20 PCs that each carry some class information, forcing random candidate predictors makes the trees much less alike, and the variance reduction from averaging outweighs the weaker individual splits; `mtry = 20` is bagging and is worst in every row. Because the selected `mtry` is the smallest value possible, the grid cannot be extended further in that direction.

In the selected forest, PC2 (0.128) and PC1 (0.095) are far more important than any other component; the next, PC3, is 0.033. The remaining 18 components all have positive importance, between 0.011 and 0.033, with no sharp cutoff, and importance is not monotone in the component index (e.g., PC11 ranks above `PC4`). Part of this flatness comes from `mtry = 1`: every split must use one randomly chosen component, so every component is used often and permuting it disrupts some splits. The two leading PCs presumably capture the main shape differences among 1, 6, and 7 (for example, a vertical stroke versus a closed loop), but this is an inference about the components, not something the importance values show directly. Importance describes how much the fitted forest's OOB accuracy depends on each component, not a causal effect, and PCA's variance ordering is not the same as an ordering by usefulness for classification.

(c) Test-set assessment

```
test_pred <- predict(fit_best, x_test)$Prediction
test_pred <- factor(test_pred, levels = levels(y_train))
table(Predicted = test_pred, Actual = y_test)
```

```
         Actual
Predicted   1   6   7
        1 102   0   0
        6   0 106   2
        7   2   0 105
```

```r
mean(test_pred != y_test)
```

```
[1] 0.0126183
```

The selected forest misclassifies 4 of 317 test digits, a test error of 0.0126, compared with the OOB estimate 0.0214. The test labels were used only in this chunk.

Question 3: Planning a random-forest analysis skill

(a) Approach

I would write the skill for a capable AI that already knows how to fit and evaluate a random forest, so I wouldn't spend instructions re-teaching textbook steps like drawing bootstrap samples or averaging trees; at most I would keep one-line reminders of the few standard steps that are cheap to state and costly to skip, such as keeping the test set untouched. The real value of a skill is in what the AI cannot infer from the task: which package version to use and its quirks, how wide the tuning grid should be and what to do when the best value lands on its edge, how to break ties, and how I want results reported and interpreted. These are conventions and preferences, not facts, so a capable model will choose something reasonable but not necessarily what I want, and not the same thing on every run. Detail can also backfire: a long, rigid procedure invites the AI to follow the wording even when the data call for something else, and it buries the instructions that actually matter among ones the model would have followed anyway. So I would use a layered structure, with a short core of essential rules first, then my methodological choices and preferences, each with a brief reason so the AI can apply the intent rather than follow the wording mechanically. My principle: specify decisions, not knowledge; tell the AI what I have chosen and why, and trust it with what any competent analyst already knows.

(b) Ten instructions, from basic to personal

Basic / standard

1. Split the data into training and test sets with a fixed seed before any analysis, and never use the test labels until the final evaluation.
2. Fit all preprocessing (scaling, imputation, PCA) on the training data only, then apply the fitted transformation to the test set.
3. Set and report the random seed, the package name and the package version; when using RLT, use the course version and set ncores = 1 (the CRAN version behaves differently).

Methodological choices

4. Tune with OOB error rather than cross-validation, using at least 500 trees, and search over mtry and node size, with mtry covering 1 to p when p is small.
5. If the best value lands on the edge of the grid, extend the grid in that direction and refit, and report whether the best value still sits on the edge.
6. Break ties in OOB error by choosing the larger node size, then the smaller mtry, and say that a tie occurred (I default to the simpler, smoother model).
7. Report the test error next to the OOB error for the final model, plus a confusion matrix for classification, and comment if the two errors differ by more than about 2 percentage points.

Personal preferences

8. Use permutation importance, not impurity importance, and state that importance is not causal and that correlated predictors split importance between them (I prefer cautious interpretation to confident claims).

9. Make exactly two figures, OOB error versus mtry with one line per node size and a sorted horizontal bar chart of the top 10 importances; label both axes, use a colorblind-safe palette, and put titles in captions rather than on the plots (clean figures read better in a short report).
10. Keep the report under two pages excluding code, show the code in an appendix, pull every number in the text from code output rather than typing it, use a plain and direct tone, and end with an AI-use disclosure (I want results I can verify and a report I can read in five minutes).